

C# REST API開発コース キャッチコピー作成プロンプト記録

キャッチコピーを決定するまでのプロンプト手順書です。同じ作業を再現する際は、以下の順序でClaudeに入力してください。

事前準備

研修コースのスライド資料（.pptx形式）を用意してください。Claudeがコースの内容・特徴・強みを把握した上でキャッチコピーを生成します。

STEP 1 | コース内容の読み込み

スライドをアップするので、読み込んでください。

スライドファイル（.pptx）をアップロードします。Claudeがテキストを抽出し、章立て・学習内容・特徴を把握します。

STEP 2 | 品質・将来性の評価

最初に、この研修コースの品質、将来性を評価してください。

コースの強み・改善点・将来性をClaudeが分析します。この評価内容がキャッチコピーの方向性の土台になります。

STEP 3 | 評価のMarkdown化と差別化の明確化

提示していただいた内容にMarkdownを作成してください。
DDDをベースにしているところは差別化できませんか？

評価内容をMarkdownファイルとして保存します。同時に、このコース最大の差別化ポイント（DDD）を明確に言語化するよう依頼します。ここで差別化ポイントを整理しておく、以降のキャッチコピーの質が上がります。

STEP 4 | 最初のキャッチコピー案の生成

この研修にキャッチフレーズをつけるならどのようなものいいでしょうか？
いくつかの案を提示してください。

複数案（5～6案）が提示されます。コースの特徴（DDD・セキュリティ・設計原則）をベースにした案が生成されます。

STEP 5 | 対象者を絞ったキャッチコピーの生成（若手エンジニア向け）

若いエンジニア層を対象と思うので、そのあたりも考慮してください。

対象者の課題感・感情・キャリア意識に寄り添った言葉に調整されます。「なんとなく動く」「先輩のコードが読めない」など、若手特有の悩みに刺さる表現になります。

STEP 6 | 対象者を絞ったキャッチコピーの生成（事業会社向け）

自社のシステムを社内で開発している事業会社などに向けたキャッチコピーをお願いします。

内製開発チームが抱える課題（外注依存・技術負債・変更耐性）に寄り添った表現になります。「仕様変更が怖くないチームをつくる」「触りたくないコードを次の世代に残さない」など、組織・チーム視点の言葉が生成されます。

STEP 7 | 汎用的なキャッチコピーの生成

ありがとうございます。いろいろな立場の方が見ると思うので、汎用的なキャッチコピーをお願いします。

特定の立場・職種に偏らず、誰が読んでも「自分ごと」として受け取れる表現に調整されます。「動くだけでなく、変えられるAPIをつくる」「設計原則が身につくと、どの現場でも通用する」など、普遍的な価値を訴求する案が生成されます。

STEP 8 | 自分のアイデアをぶつけてみる

例えばですが、設計原則が身につくAPI開発ハンズオンというのでどうでしょうか？

自分のアイデアをそのままClaudeにぶつけます。Claudeがそのまま使う案・少し磨いた案を複数提示してくれます。ここで「これが近い」と感じたものをベースに最終調整します。

STEP 9 | 確定

設計原則が身につくREST API開発ハンズオンに決めました。

確定したキャッチコピー

設計原則が身につくREST API開発ハンズオン

選定理由：

- シンプルで誰にでも伝わる
- 「設計原則が身につく」でDDDをベースにした差別化が一言で表現できる
- 「REST API開発ハンズオン」で研修内容と形式が明確に伝わる
- 若手エンジニア・事業会社・Slerなど、どの立場からでも違和感なく受け取れる

ポイント・コツ

ポイント	内容
評価→差別化→コピーの順に進める	いきなりコピーを作るより、まず強みを整理してからの方が質が上がる
対象者ごとに案を出してもらう	若手・事業会社・汎用と段階的に絞ることで、最終的に「誰にでも刺さる言葉」に辿り着きやすい
自分のアイデアをぶつける	「〇〇というのはどうか」と投げると、Claudeがバリエーションを展開してくれる
形式説明になっていないか確認する	「ハンズオン」などの形式語は内容説明になりやすい。差別化の言葉と組み合わせる
最後は自分で決める	複数案の中から「これだ」と思うものを選ぶのは人間の判断。Claudeは選択肢を広げる役割

全体の流れまとめ

- STEP 1 | スライドをアップロードして内容を把握させる
- STEP 2 | 品質・将来性の評価を依頼する
- STEP 3 | 評価をMarkdown化し、差別化ポイントを明確にする
- STEP 4 | 最初のキャッチコピー案を複数生成する
- STEP 5 | 若手エンジニア向けに調整する
- STEP 6 | 事業会社（内製開発）向けに調整する
- STEP 7 | 汎用的な表現に調整する

STEP 8 | 自分のアイデアをぶつけてバリエーションを出す
STEP 9 | 最終案を確定する

C# REST API開発コース 研修概要作成プロンプト記録

「本研修について」「本研修の3つの目的」を再現するためのプロンプト手順書です。Claudeに対して、以下の順序で入力してください。

事前準備

以下を用意してください。

- 研修コースのスライド資料（.pptx形式）
- 確定したキャッチコピー（文章のトーンの基準になります）

本コースの確定キャッチコピー：「設計原則が身につくREST API開発ハンズオン」

SECTION 1 | 本研修について

STEP 1 | 文章の生成

それでは次に、研修概要の文書作成をお願いします。

スライドをアップロード済みであればそのまま送信します。Claudeがコース内容をもとに複数のトーンで文章案を提示します。

STEP 2 | 案の選択

案Aにします。

提示された案（A / B / C）から選択します。今回は案A（設計力の習得を前面に出したトーン）を選択しました。

確定文（案A）

本研修は、C#および.NETを用いたREST API開発を、ドメイン駆動設計（DDD）をベースに体系的に学ぶハンズオンプログラムです。

「動くコードが書ける」から一歩進み、変更に強く・テストしやすく・チームで共有できる設計を身につけることを目的としています。ドメイン層・インフラストラクチャ層・アプリケーション層・プレゼンテーション層の4層アーキテクチャを実装しながら、依存性逆転の原則（DIP）・Repositoryパターン・Unit of Workなどの設計原則を体得します。

さらにJWT認証・HTTPS・CORS・HttpOnly Cookieを第二部として独立した章で扱い、セキュリティを考慮した実践的なREST APIの構築まで一気通貫で学びます。

SECTION 2 | 本研修の3つの目的

STEP 1 | 文章の生成

次は、本研修の3つの目的をお願いします。

「本研修について」で選んだ案のトーンを引き継いで文章案が生成されます。

STEP 2 | 案の選択

案Bにします。

今回は案B（「できるようになること」視点で整理した受講者目線型）を選択しました。

確定文（案B）

本研修では、以下の3つを習得することを目的としています。

- 1. 変更に強い設計ができるようになる** DDDの4層アーキテクチャと設計原則（DIP・Repository・Unit of Work）を実装を通じて体得し、仕様変更・技術変化に強いコードを書けるエンジニアを目指す。
- 2. REST APIを設計から実装まで自走できるようになる** ASP.NET Coreの主要機能を活用し、エンドポイント設計・ドメイン実装・例外処理・OpenAPI/Swagger対応まで、一貫したREST APIを独力で構築できる力を養う。
- 3. セキュリティを自分の手で実装できるようになる** 「知っている」を「できる」に変える。JWT認証・HTTPS・CORS・HttpOnly Cookieを実際に実装することで、セキュリティを他任せにしないエンジニアへの第一歩を踏み出す。

ポイント・コツ

ポイント	内容
キャッチコピーを先に確定させる	コピーのトーンが文章全体の方向性の軸になるため、先に確定しておくとう一貫性が出る
セクションごとに1つずつ進める	まとめて依頼するより、1セクションずつ確認しながら進める方が精度が高い
トーンを統一しなくてよい	「本研修について」と「目的」で異なる案（A・B）を選んでも問題ない。内容が補完し合う

ポイント	内容
受講者視点 vs 技術視点で使い分ける	概要文は技術軸（案A）、目的文は受講者視点（案B）のように役割で使い分けると読みやすくなる
スライドは最初に一度読み込めばよい	同じ会話内であれば、再アップロードは不要

全体の流れまとめ

STEP 1 | スライドをアップロードして内容を把握させる

STEP 2 | 「研修概要の文書作成をお願いします」と指示する

STEP 3 | 「本研修について」の案を選択する

STEP 4 | 「本研修の3つの目的をお願いします」と指示する

STEP 5 | 「目的」の案を選択する

STEP 6 | 続けて次のセクション（到達目標・前提知識・受講環境）へ進む

STEP 7 | 全セクション確定後にプロンプト記録をまとめて作成する

こんな企業にお勧めします 作成プロンプト記録

「こんな企業にお勧めします」セクションを再現するためのプロンプト手順書です。Claudeに対して、以下の順序で入力してください。

事前準備

以下を用意してください。

- 研修コースのスライド資料（読み込み済みであれば再アップロード不要）
- 確定したキャッチコピーと「本研修について」の文章（トーンの基準になります）

STEP 1 | 文章の生成

次は、こんな企業にお勧めしますを作成したいです。

Claudeがコース内容をもとに複数のトーンで文章案を提示します。今回は以下の3案が提示されました。

- 案A：課題起点型（企業の悩みに寄り添う）
- 案B：企業タイプ型（業種・フェーズを示さず特徴で分類）
- 案C：シンプル箇条書き型（コンパクトに一覧化）

STEP 2 | 案の検討

案Cがいい気がするのですが？

迷ったときは「〇〇がいい気がするのですが？」と相談するかたちでもOKです。 Claudeが選択を後押しするコメントをくれます。

STEP 3 | 案の確定

案Cで確定します。

確定文（案C）

本研修は、以下のような企業にお勧めです。

- C#・ASP.NET CoreによるREST API開発チームの設計力を高めたい
- DDD・設計原則を実装レベルで習得させたい
- セキュリティを考慮したAPI実装をチーム全体に浸透させたい
- 技術負債を解消し、長く保守できるシステムを構築したい

ポイント・コツ

ポイント	内容
業種・企業タイプを示す言葉を含めない	「Sler」「事業会社」などの業種を明示すると対象が絞られすぎる。課題・状況の言葉だけで表現する方がどの立場の読者にも刺さりやすい
シンプルな箇条書きが汎用性が高い	説明が長くなるほど「自分の会社のことではない」と感じられやすい。簡潔な表現が広く刺さる
「〇〇がいい気がするのですが？」は有効	迷ったときに相談するかたちで入力すると、Claudeが選択を後押しするコメントをくれる
項目数は4つ程度が読みやすい	多すぎると印象が薄れる。優先度の高い課題に絞り込む

全体の流れまとめ

STEP 1 | 「こんな企業にお勧めしますを作成したいです」と指示する
STEP 2 | 複数案を見て、気になる案を「〇〇がいい気がするのですが？」と相談する
STEP 3 | 「案〇〇で確定します」と確定する

必要な受講環境 作成プロンプト記録

「必要な受講環境」セクションを再現するためのプロンプト手順書です。Claudeに対して、以下の順序で入力してください。

事前準備

以下を用意してください。

- 研修コースのスライド資料（読み込み済みであれば再アップロード不要）
- 演習環境のスライド（IDE・OS・SDK・DBのバージョンが記載されているもの）

スライドに記載されている演習環境の情報をもとにClaudeが表を生成します。事前にDBの種類・バージョンなど正確な情報を確認しておくで修正が少なくなります。

STEP 1 | 文章の生成

必要な受講環境を作成します。

Claudeがスライドの演習環境をもとに複数の構成で文章案を提示します。今回は以下の2案が提示されました。

- 案A：カテゴリ別箇条書き型
- 案B：テーブル型（区分・環境/ツール・役割で整理）

STEP 2 | 誤りの修正

すいません。DBはPostgreSQLに変更してください。

スライドの内容と異なる箇所があった場合は、この段階で修正を依頼します。具体的に「〇〇を〇〇に変更してください」と伝えると確実に反映されます。今回はMySQLをPostgreSQLに、DBクライアントをpgAdmin 4に修正しました。

STEP 3 | 案の選択

案Bにします。

今回は案B（テーブル型）を選択しました。区分・環境/ツール・役割の3列構成で、一覧性が高く読みやすい形式です。

STEP 4 | Markdownファイルへの出力

お願いします。

確定文（案B）

区分	環境 / ツール	役割・用途
ホストOS	Windows 11	研修用PCの基本環境
IDE	Visual Studio Code	コード編集・ターミナル 操作・デバッグ
実行環境	WSL2（Ubuntu 24.04）	Linux環境でのアプリケー ション実行
SDK	.NET 8 LTS	C# / ASP.NET Core アプ リケーション開発・実行
RDBMS	PostgreSQL	サンプル・演習アプリの データベース
SQLツール	pgAdmin 4	データベースの接続・操 作・確認
VSCode拡張機能	.NET Install Tool / C# Dev Kit / C# / [Extension Pack] C# / C# Namespace Autocompletion / C# XML Documentation Comments / C# Snippets	C# / .NET 開発の補完・ デバッグ・ドキュメント 支援

ポイント・コツ

ポイント	内容
DBの種類・バージョンは事前に確認 する	スライドの記載と実際の環境が異なる場合があるため、事前に 正確な情報を把握しておく
テーブル型が汎用性が高い	箇条書きより区分・ツール・役割が一目でわかるテーブル型の 方が読みやすく、資料としての完成度が高い
修正は「〇〇を〇〇に変更してくださ い」と具体的に伝える	曖昧な表現より具体的な指示の方が確実に反映される
VSCode拡張機能はまとめて1行にでき る	拡張機能が多い場合は1セルにまとめて記載するとテーブルがコ ンパクトになる

全体の流れまとめ

STEP 1 | 「必要な受講環境を作成します」と指示する
STEP 2 | スライドの内容と異なる箇所があれば修正を依頼する

STEP 3 | 提示された複数案から選択する（箇条書き vs テーブル）

STEP 4 | 「お願いします」と伝えてMarkdownファイルに保存する